Applied Generative AI Engineering

Session 5 — Introduction to Retrieval-Augmented Generation (RAG)

Instructor: Shrey

Module 3 — RAG Architecture & Vector Search Systems

This session begins a new module focused on building intelligent systems that can reason over private organisational knowledge — a foundational capability in modern AI engineering.

Course Progression

Here is where we stand in the overall programme — and where we are headed.

Module 1
AI Foundations

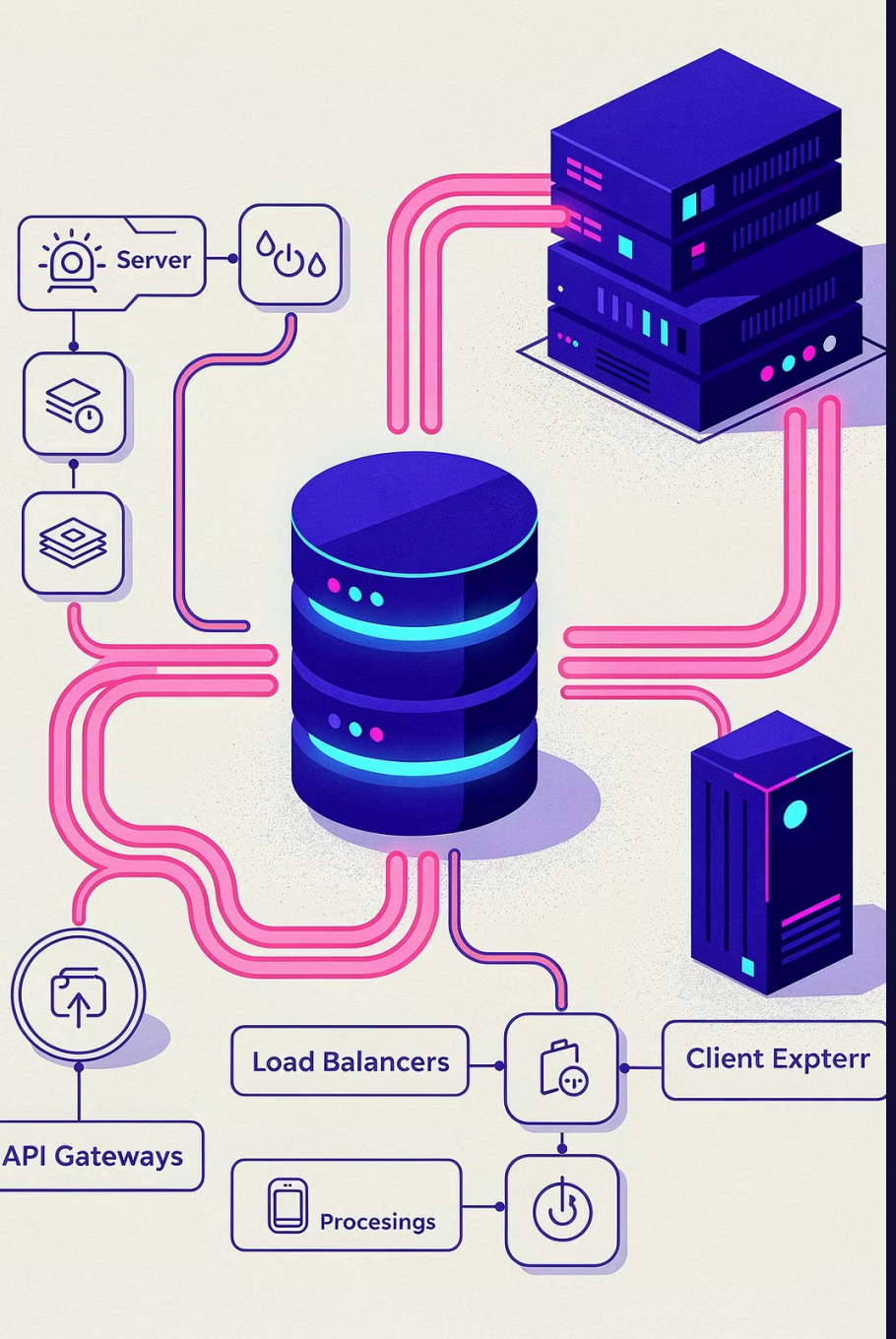
Module 2
Python API Middleware

Module 3 ← Today
RAG Systems

Module 4
RAG Optimisation

Modules 5–7
Fine-tuning · Agentic · Cloud · Capstone

Today marks the start of Module 3. RAG Systems is the most architecturally rich module in the course — every concept here feeds directly into the modules that follow.



Recap: What We Built So Far

In Sessions 1–4 you assembled a production-style AI backend. That system is now the foundation on which RAG will be layered.

System Architecture

01

Client Request

02

FastAPI Backend

03

LLM Gateway

04

LLM Providers

Capabilities Achieved

- Provider-agnostic LLM Gateway
- Modular service architecture
- Multiple LLM provider integration
- Environment variable configuration
- Structured logging and error handling

This backend will now become the **foundation for RAG systems** - we extend it rather than replace it.

The Missing Capability

Can our backend answer questions about private company documents?

The LLM Gateway we built can call any language model — but every model we call has a hard boundary: its training data. Anything outside that boundary is invisible to it.

Internal Policy Docs

HR policies, compliance rules, internal SOPs that were never in any training corpus.

Legal Contracts

Confidential agreements, vendor contracts, and legal terms specific to the organisation.

Company Knowledge Bases

Engineering wikis, runbooks, and internal documentation built up over years.

Support Manuals

Customer service scripts, troubleshooting guides, and product FAQs.

The Enterprise Knowledge Problem

Organisations accumulate enormous volumes of structured and unstructured knowledge — none of which was ever included in a public LLM's training run. This is the core problem RAG is designed to solve.

PDFs

Reports, contracts, research papers

Wikis

Confluence, Notion, internal docs

Databases

Structured records and tables

Knowledge Bases

FAQs, runbooks, product docs

Key insight: LLMs cannot access private organisational knowledge at inference time. They only know what they were trained on — and they were never trained on your organisation's data.

Why Naïve Solutions Fail

Before examining RAG, it is worth understanding why the obvious approaches break down at scale.

Fine-tune the Model

Train the LLM on all company documents. **Problems:** extremely expensive, slow to update when documents change, and requires a full retraining cycle for every knowledge update.

Stuff Entire Documents into Prompts

Pass all relevant documents as context with each request. **Problems:** context windows have hard token limits, costs scale linearly with document size, and relevance degrades with very long contexts.

Manual Search + Summarisation

Have a human retrieve the relevant document, then pass it to the LLM. **Problems:** not scalable, introduces latency, and relies on human judgement for every query.

Each naive approach trades one problem for another. What is needed is a **systematic architectural solution**.

Introducing Retrieval-Augmented Generation

> **Retrieval-Augmented Generation (RAG)** combines information retrieval systems with language models to enable dynamic, knowledge-grounded response generation.

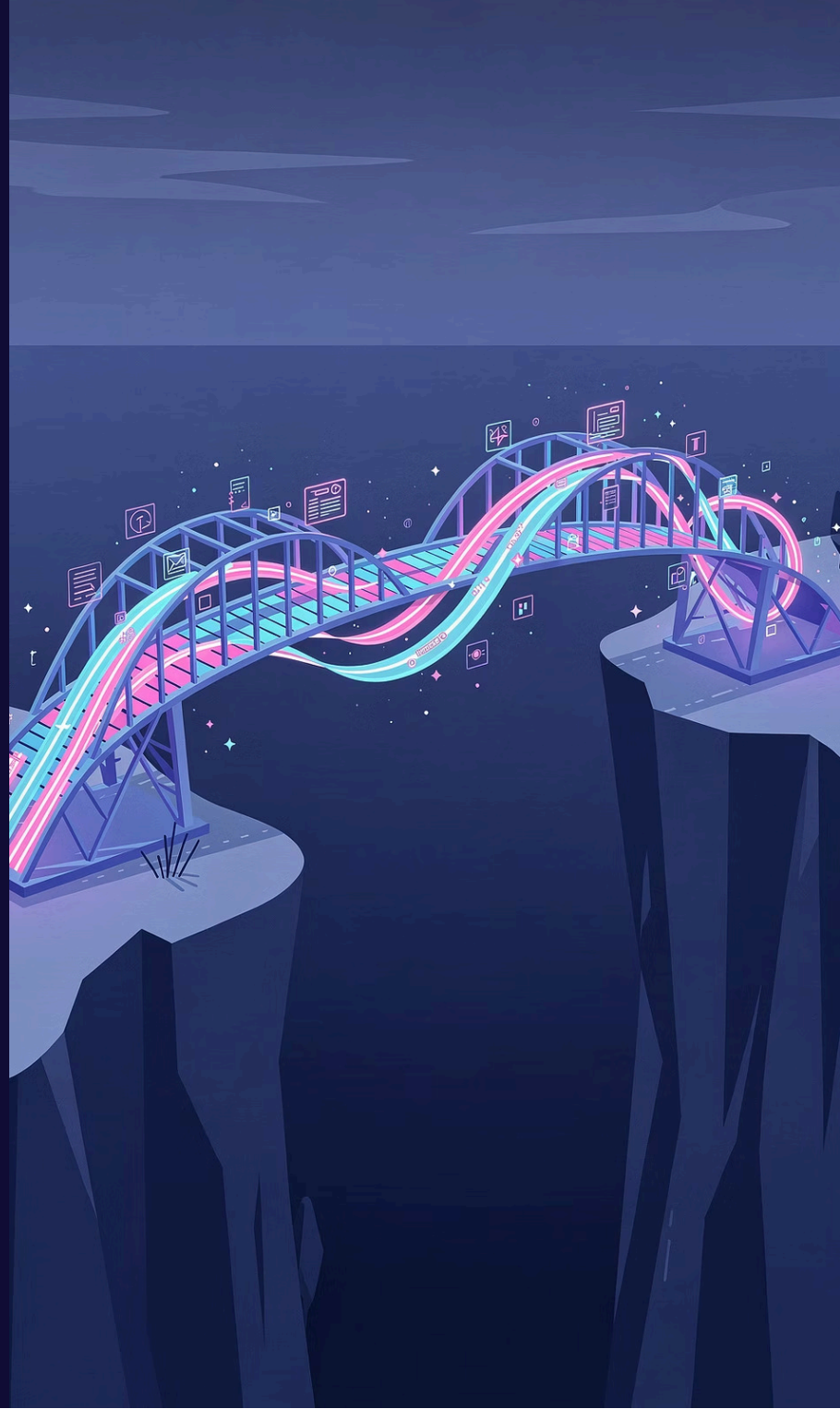
The Core Insight

Instead of baking all knowledge into model weights at training time, RAG retrieves only the *relevant* knowledge at *query time*. The LLM then uses this retrieved context to generate a grounded, accurate answer.

This separates **knowledge storage** from **language generation** — making systems easier to update, cheaper to run, and more accurate on domain-specific tasks.

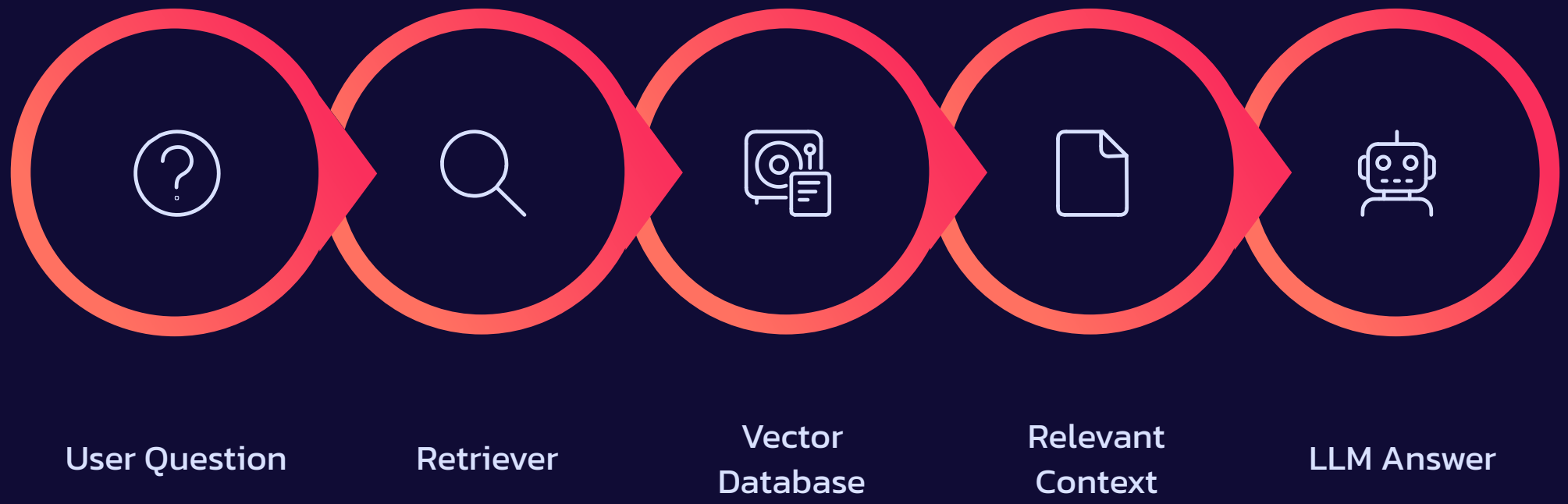
What Changes

- Knowledge lives in a **vector database**, not model weights
- Retrieval happens **at inference time**
- LLM generates answers **grounded in retrieved context**
- Knowledge can be **updated without retraining**



High-Level RAG Architecture

At the highest level, RAG inserts a retrieval step between the user's question and the language model. The LLM never answers from memory alone.



The critical principle: **retrieval always happens before generation.** The LLM is handed relevant evidence; it does not need to recall facts from its weights. This architectural separation is what makes RAG systems reliable on private knowledge.

Two Pipelines in Every RAG System

RAG systems operate through two distinct pipelines that run at different times. Understanding this separation is essential for system design.

Offline Pipeline

Runs once, or when documents change

01

Raw Documents

02

Chunking

03

Embedding Generation

04

Vector Database Index

Online Pipeline

Runs on every user query

01

User Query

02

Query Embedding

03

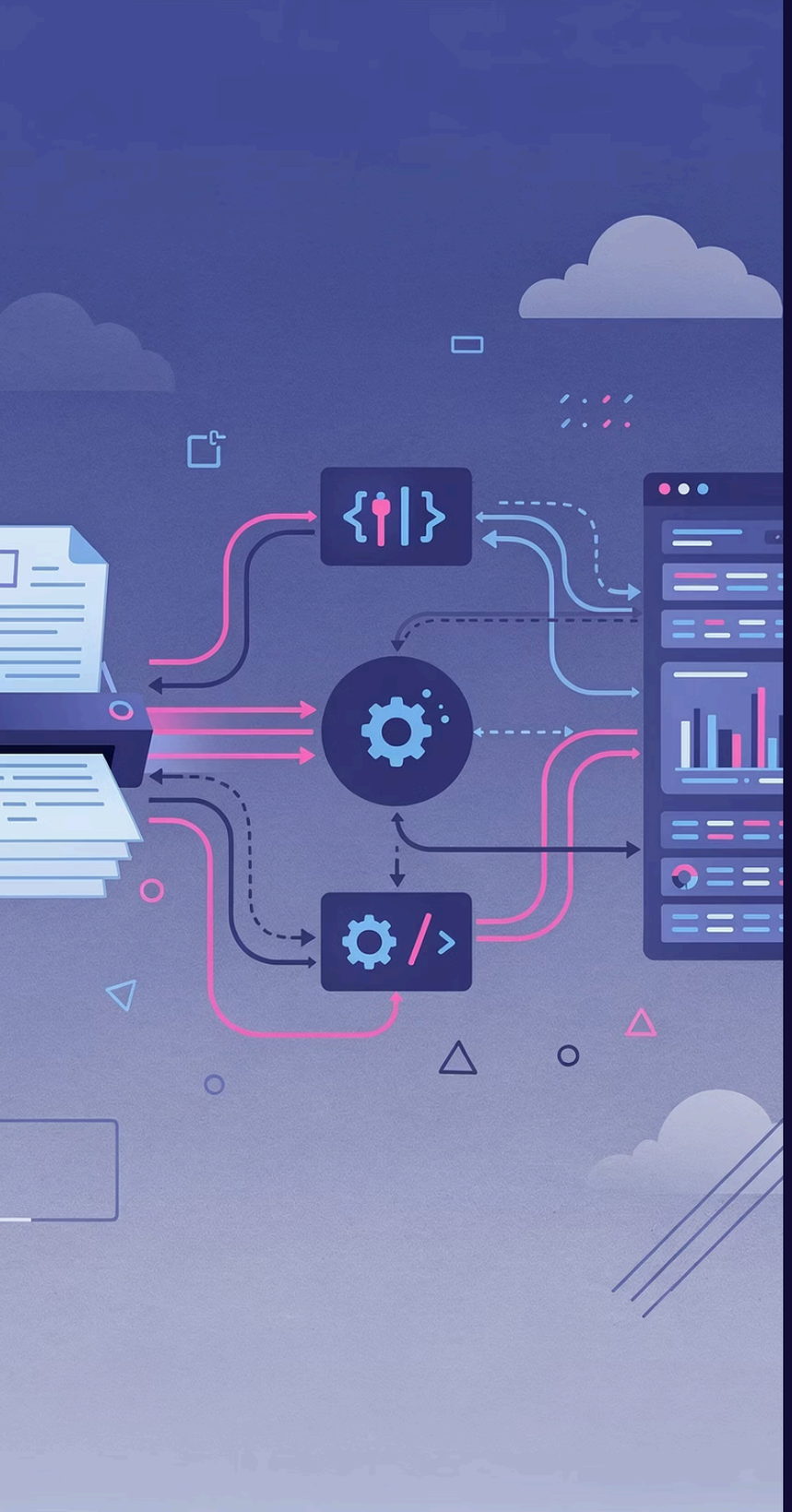
Vector Similarity Search

04

Context Retrieval

05

LLM Response



Document Ingestion

The offline pipeline begins with ingestion — converting raw, heterogeneous documents into a form the system can work with. The goal is a **structured knowledge representation** regardless of source format.

PDFs

Research papers, reports, policy documents

Markdown / HTML

Documentation sites, wikis, engineering runbooks

Databases

Structured records exported as text for indexing

Support Logs

Historical ticket data, Q&A pairs, conversation logs

Each format requires a parser. The output is always clean text — the common currency of the ingestion pipeline.

Why Documents Must Be Split: Chunking

The Problem with Large Documents

LLMs have finite context windows. Passing an entire 100-page PDF into every prompt is impossible — and even if it were possible, it would be prohibitively expensive and retrieval would be imprecise.

The Solution: Chunking

Documents are divided into smaller, self-contained pieces called **chunks**. Each chunk becomes an independent unit that can be embedded, indexed, and retrieved on its own merits.

Chunking in Practice

1. **A 100-page PDF**

   May produce hundreds of individually searchable chunks

2. **Chunk size matters**

   Too small → loss of context. Too large → poor retrieval precision.

3. **Overlap is common**

   Adjacent chunks share a small window of text to preserve boundary context.

Embeddings Explained

> An **embedding** is a dense numerical vector that encodes the semantic meaning of a piece of text. Similar meanings produce vectors that are close together in high-dimensional space.

What Happens

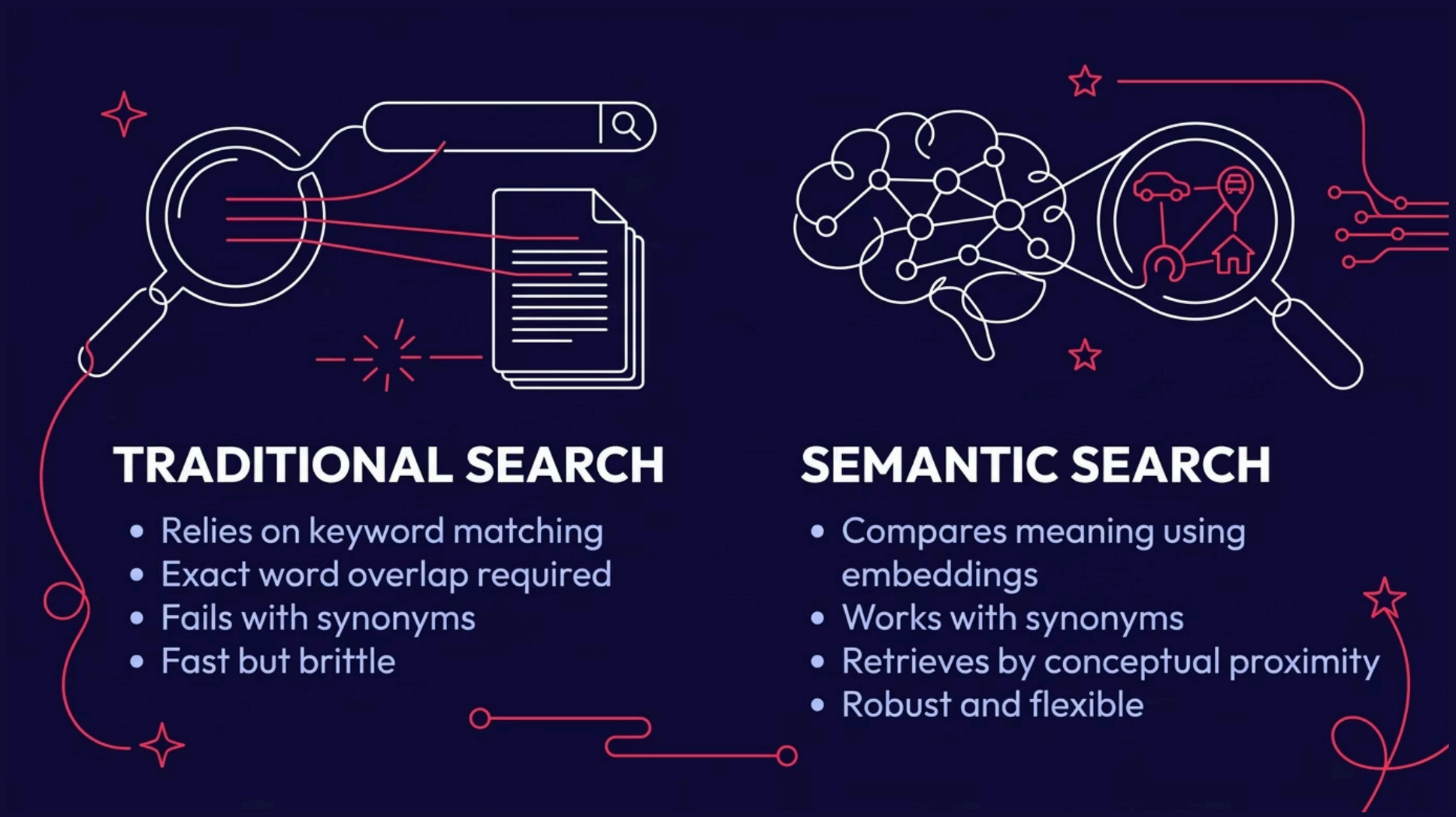
An embedding model reads a chunk of text and outputs a fixed-length array of floating-point numbers — typically 768 to 3,072 dimensions. The geometry of this space is learned during training, so semantically related concepts cluster together.

Why It Matters for RAG

- Every document chunk is converted into an embedding
- Every user query is converted into an embedding
- Retrieval becomes a **geometry problem**: find the chunks whose vectors are closest to the query vector
- No keyword overlap required — meaning drives matching

Semantic Search vs. Traditional Search

Understanding this distinction is central to understanding why RAG works where keyword search fails.



Example: a query for *"how do I reset my password?"* can semantically retrieve a document titled *"Account Recovery Procedures"* — even with zero word overlap. Keyword search would miss this entirely.

Vector Similarity

Once text is encoded as vectors, retrieval is a matter of finding the nearest neighbours in high-dimensional space.

How Similarity Works

Embeddings exist in a continuous vector space. Texts with similar semantic content are geometrically close. The retriever computes a distance or similarity score between the query vector and every stored document vector, then returns the closest matches.

Common Similarity Metrics

Cosine Similarity

Measures the angle between two vectors. Most common in NLP — direction matters more than magnitude.

Dot Product

Magnitude-sensitive. Used when vector norms carry meaning (e.g., in some biencoder models).

Euclidean Distance

Straight-line distance in space. Less common for NLP but used in some approximate nearest-neighbour systems.

Vector Databases

A vector database is a purpose-built storage and retrieval system optimised for high-dimensional embeddings. Unlike traditional databases that query rows and columns, vector databases execute **approximate nearest-neighbour (ANN) searches** at scale.

Pinecone

Fully managed, cloud-native. Easy to integrate and scale. Popular in production RAG deployments.

Chroma

Open-source, developer-friendly. Ideal for local development and prototyping RAG pipelines.

Weaviate

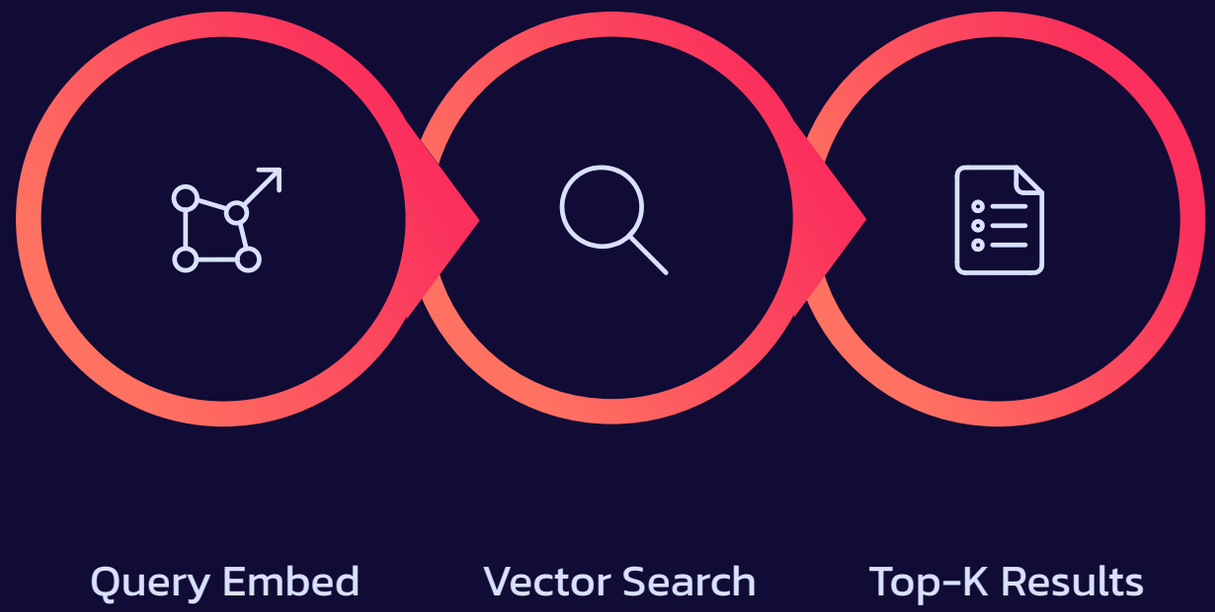
Open-source with hybrid search (vector + keyword). Strong schema and multi-tenancy support.

pgvector

PostgreSQL extension. Adds vector search to a familiar relational database — minimal new infrastructure required.

The Retrieval Step: Top-K Search

When a user submits a question, the online pipeline executes a precise, three-step retrieval operation.



What is K?

K is the number of document chunks returned by the retriever — typically 3 to 10. A larger K provides more context but increases prompt size and token cost. Selecting the right K is a tuning decision made during RAG optimisation.

What the Retriever Returns

Each retrieved result includes the raw chunk text and a similarity score. The retriever itself has no knowledge of language — it is a pure geometry engine operating on vectors.

Context Injection: Grounding the LLM

Retrieved chunks are inserted into the prompt before the user's question. This is called **context injection** — and it is the mechanism that grounds the LLM in your organisation's knowledge.

1

System Instruction

Tells the model to answer only using the provided context and to say "I don't know" if the answer isn't present.

2

Retrieved Context

The Top-K document chunks most relevant to the user's query, injected verbatim or lightly formatted.

3

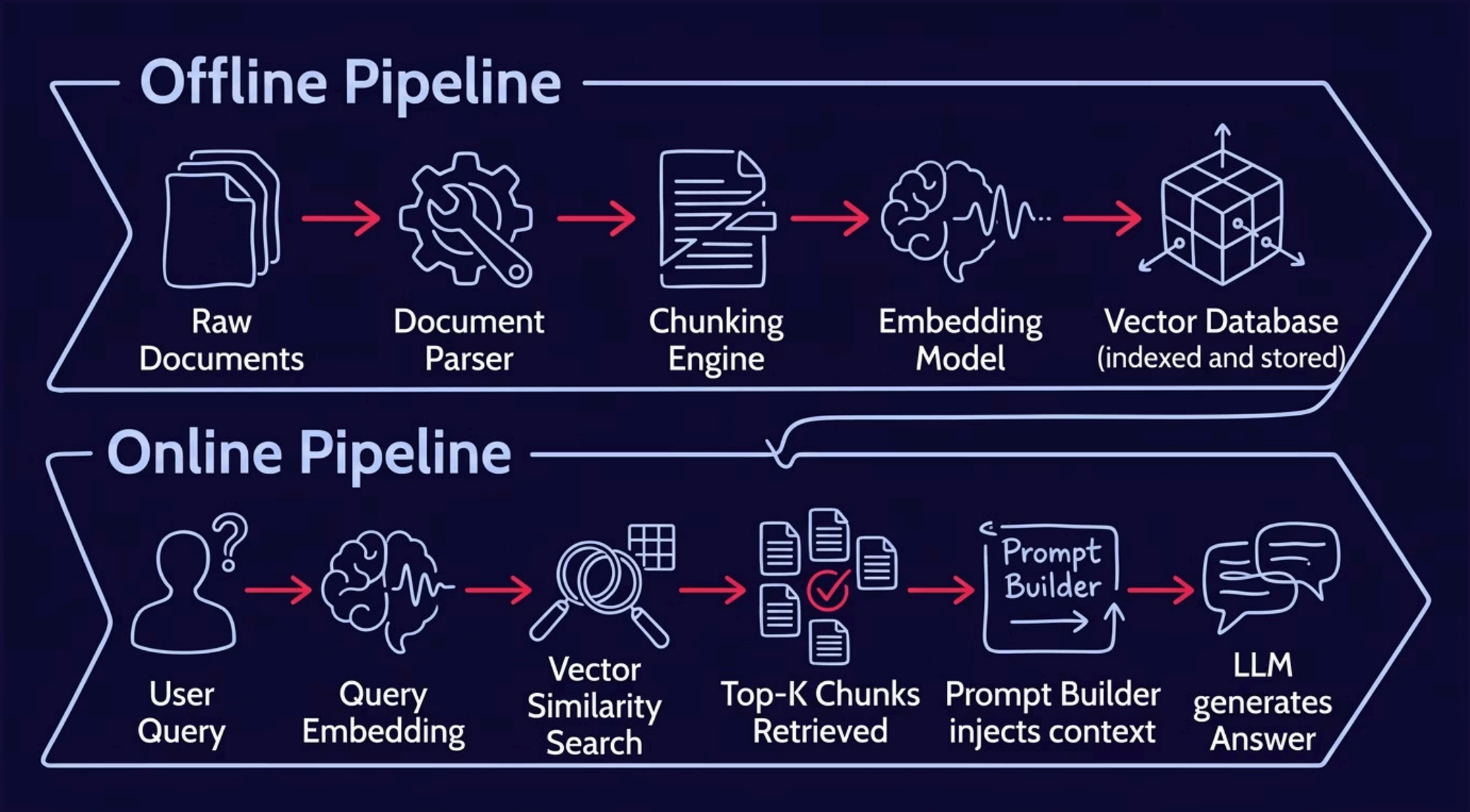
User Question

The original query from the user, appended after the context so the model can answer against it.

Design principle: The LLM is not asked to recall — it is asked to *read and respond*. The quality of the answer is therefore bounded by the quality of the retrieval.

The Complete RAG Pipeline

Bringing both pipelines together — this is the full end-to-end architecture you will implement in the upcoming sessions.



Every component in this diagram is a distinct engineering concern with its own design decisions. The upcoming sessions address each in turn — today's goal is to hold the **complete mental model** before touching any implementation.

Real-World RAG Applications

RAG is not a research prototype — it underpins a large and growing class of production AI products across industries.

Enterprise Knowledge Assistants

Internal chatbots that answer employee questions against company wikis, HR policies, and documentation.

Customer Support Chatbots

Deflect tier-1 support tickets by retrieving answers from product manuals and known issue databases.

Legal Research Tools

Search across thousands of case files, contracts, and regulatory documents with semantic precision.

Medical Document Assistants

Retrieve relevant clinical guidelines and patient records to support diagnosis and treatment planning.

What We Build Next

Project 2 — Enterprise RAG Knowledge Assistant

The conceptual foundation is in place. Starting next session, you will implement each stage of the pipeline from scratch — building a production-grade RAG system on top of the FastAPI backend from Project 1.

Step 1 — Document Ingestion

Build parsers for PDF and Markdown sources; normalise to clean text.

Step 2 — Chunking Pipeline

Implement configurable chunk size and overlap strategies.

Step 3 — Embedding Generation

Integrate an embedding model to convert chunks into vectors.

Step 4 — Vector Indexing & Retrieval

Persist embeddings to a vector database; implement Top-K similarity search.

Step 5 — AI-Generated Answers

Inject retrieved context into prompts and stream grounded responses via the LLM Gateway.